

Ejercicio: Implementando a los booleanos

Queremos modelar el álgebra de Boole. Para hacerlo, tenemos que representar a los distintos *valores veritativos* (también llamados *booleanos* en este contexto) como objetos. En general hay numerosas estructuras algebraicas que son álgebras de Boole; nosotros vamos a tomar como referencia a una en particular, la *lógica binaria*, que utiliza y define relaciones entre *dos* objetos: *verdadero* y *falso*.

En particular, queremos representar las siguientes operaciones:

- Negación: `!unBooleano` es el booleano opuesto a `unBooleano`.
- Conjunción: `unBooleano && otroBooleano` es *verdadero* si ambos son *verdadero* y *falso* en cualquier otro caso.
- Disyunción: `unBooleano || otroBooleano` es *verdadero* si al menos uno es *verdadero* y *falso* si ambos son *falso*.
- Disyunción exclusiva: `unBooleano xor otroBooleano` es *verdadero* si sólo uno de los booleanos es *verdadero*, y *falso* si ambos o ninguno es *verdadero*.

Se pide:

Implementar el modelo de lo pedido en Scala, modelando a *verdadero* y *falso* como **objetos nuestros** (es decir, no usar los objetos predefinidos de Scala `true` o `false`). Una consecuencia de esto es que *ninguna prueba* debería mencionar a `true` o `false`.

Para modelar las operaciones se pueden usar mensajes que reciban los booleanos; por ejemplo `unBooleano.negado()` para representar la negación de `unBooleano`, o `unBooleano.y(otroBooleano)` para la conjunción.

El ejercicio deberá incluir, como siempre, el código de la solución así como las pruebas automatizadas correspondientes.



Puntos bonus

1. Asegurarse de que *la implementación* no haga referencia a `true` ni `false` (booleanos nativos de Scala) de manera interna, ni utilice `ifs`.
2. Utilizar notación de operadores para los mensajes; es decir, los mensajes tienen que leerse igual que en la especificación (por ejemplo: `unBooleano && otroBooleano` para la conjunción, o `!unBooleano` para la negación)¹.

¹ Se puede leer más sobre esto en <https://docs.scala-lang.org/tour/operators.html> o en la sección 5.4* del libro *Programming in Scala* (Odersky et al., 2021). Un caso particular “raro” es el operador `!` (para la negación), que tiene la particularidad de aplicarse de manera *prefija*. Para implementarlo como mensaje se tiene que definir un método cuyo selector es `unary_!`.

* La sección 5.4 se llama “operators are methods”. Esto, según nuestro vocabulario para objetos, es discutible. Quizás debería haberse llamado “operators are messages”... para pensar.

3. Modelar a *verdadero* y *falso* como “singleton objects” (ver <https://docs.scala-lang.org/es/tour/singleton-objects.html>). Comparar esto con la implementación que tenían hasta el momento, ¿tiene sentido modelarlos así? ¿Qué ventajas/desventajas le encontramos?
4. Implementar una operación que modele al `if` como un mensaje del estilo `unBooleano.ifTrue(cosa)`, en donde `cosa` es una función² que se debería ejecutar solo si `unBooleano` es *verdadero*. De manera análoga, debería haber un mensaje `ifFalse(...)`.

Por ejemplo, en un programa para un dominio particular, podríamos usarlo así:

```
saldoInsuficiente.ifTrue(() =>
  throw new RuntimeException("el saldo es insuficiente")
)
```

5. Modificar los métodos implementados en el punto (4) para poder encadenar los mensajes; es decir, queremos poder escribir por ejemplo:
`unBooleano.ifTrue(thenFunction).ifFalse(elseFunction)`
6. En Scala, un bloque (delimitado entre llaves) es una expresión cuyo valor coincide con el de su última subexpresión. Además, se puede *diferir* la evaluación de una expresión utilizando un mecanismo llamado “by-name parameters”³. Se pide modificar los mensajes implementados en los puntos (4) y (5) para que reciban un bloque en lugar de una función, manteniendo el comportamiento esperado.

Por ejemplo, queremos tener:

```
unBooleano.ifTrue {
  // hacer algo
}
```

en lugar de:

```
unBooleano.ifTrue(() =>
  // hacer algo
)
```

7. Modificar los mensajes binarios de todos los puntos anteriores para que sólo se evalúen las sub-expresiones cuando sea necesario. Por ejemplo `Falso && { throw new RuntimeException() }` (o, equivalente en este caso, `Falso && ???`) no debería lanzar una excepción y debería evaluarse a `Falso`.

² Más específicamente, una función de tipo `() => Unit`. Para ver cómo se definen funciones en Scala se puede consultar <https://docs.scala-lang.org/tour/basics.html#functions> y <https://docs.scala-lang.org/scala3/book/fp-functions-are-values.html>.

³ Se puede leer sobre esto en <https://docs.scala-lang.org/tour/by-name-parameters.html> o en las secciones 9.4 (writing new control structures) y 9.5 (by-name parameters) del libro *Programming in Scala* (Odersky et al., 2021).

Referencias

Odersky, M., Spoon, L., Venners, B., & Sommers, F. (2021). *Programming in Scala: A comprehensive step-by-step guide* (5th ed.). Aritma.